

Универзитет у Београду
Факултет организационих наука
Катедра за софтверско инжењерство



Семинарски рад из предмета
Напредне јава технологије

**Веб апликација за позајмљивање књига из
библиотеке у Spring Boot и React окружењу**

Ментор: проф.др Татјана Стојановић

Студент: Лука Ђокић

Број индекса: 2022/0312

Београд, 2026.

Садржај

1. Тема пројекта	3
2. Концептуализација	4
2.1 Концептуални модел	4
2.2 Списак функционалности	5
3. Спецификација	7
3.1 Архитектура система	7
3.2 Архитектурни обрасци	7
3.3 Технолошки склоп	7
3.4 Структура backend пројекта	8
3.5 API руте	8
3.5 Додатна библиотека	9
4. Имплементација	10

1. Тема пројекта

Пројекат носи назив Е-Библиотека и настао је из сасвим практичне потребе — да се свакодневни рад библиотеке пренесе у дигиталну форму. Ако сте икада ишли у библиотеку, знате о чему се ради: чекање на реду, библиотекар ручно листа евиденцију, тражи је ли књига слободна, записује ко је шта узео и до кад мора да врати. Овај пројекат замењује тај цео процес функционалном веб апликацијом.

Иза свега стоји REST API прављен у ASP.NET Core-у, а кориснички интерфејс је направљен у React-у. Систем препознаје два типа корисника - библиотекаре и чланове — и свакоме даје другачија права и могућности.

Члан библиотеке може из куће да прегледа каталог, провери да ли је нека књига тренутно слободна или је код неког другог члана, и да пошаље захтев за позајмицу. Не мора да долази у библиотеку само да би видео да ли је нешто доступно. Са друге стране, библиотекар добија јасан преглед свих активних позајмица, резервација на чекању и листу чланова и може да одобри захтев неколико кликова.

Једина разлика у односу на класичан начин рада је и то шта се дешава кад члан хоће да позајми више књига одједном. Раније би морао да прође кроз читав поступак засебно за сваку. Сада шаље један захтев у коме набраја све књиге које жели, и библиотекар то одобрава одједном. Ово је решено уношењем посебне асоцијативне класе *Stavkalzdavanja*, која повезује издавање са листом књига у релацији многи-на-многи.

Поред тога, жанр књиге је издвојен из обичног текстуалног поља у засебан ентитет. То значи да је жанр сада структуриран — дефинисан као *enum* са унапред познатим вредностима (роман, драма, биографија и слично), уместо да се у базу уписује произвољни текст. Ово спречава грешке као нпр. уносити 'Романи' и 'Роман' као два различита жанра.

Аутентификација је урађена кроз JWT токене — корисник се пријави, добије потписани токен и сваки следећи захтев ка серверу носи тај токен. Сервер га верификује и тако зна ко шаље захтев и да ли сме то да ради. Лозинке нигде нису сачуване у отвореном облику — хешоване су кроз BCrypt алгоритам.

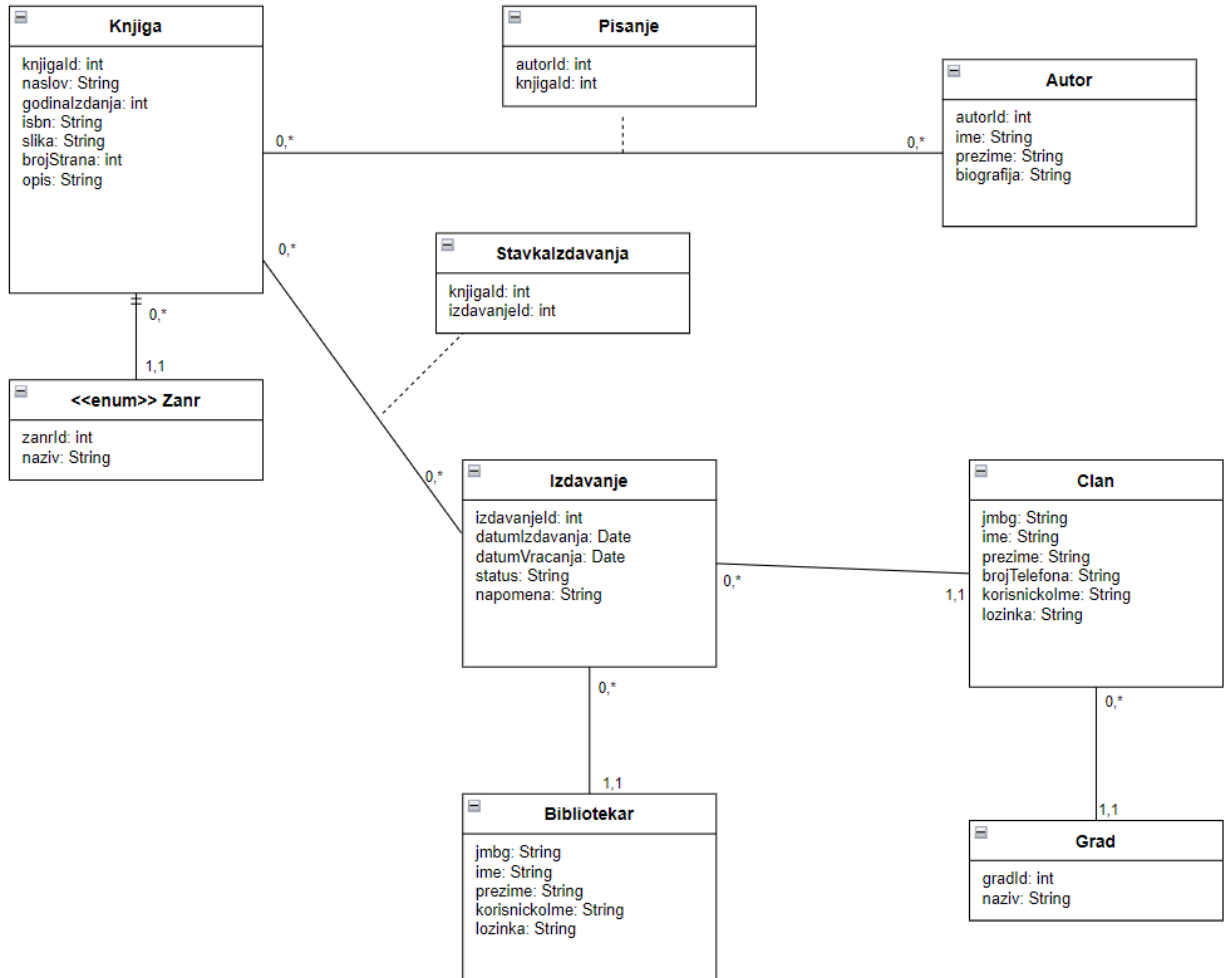
Укратко, циљ пројекта је да рад библиотеке буде бржи, прегледнији и доступан откуд год да сте.

2. Концептуализација

2.1 Концептуални модел

Систем је моделован следећим ентитетима:

- **Knjiga** — основни ентитет система; садржи наслов, ISBN, годину издања, слику, број страна и опис. Повезана је са жанром и ауторима.
- **Zanr** — издвојен из текстуалног поља у засебан enum ентитет са ID-ем и називом.
- **Autor** — садржи име, презиме и биографију. Повезан са књигама преко асоцијативне класе Pisanje.
- **Pisanje** — асоцијативна класа за везу M:N између аутора и књига (чува autorId и knjigId).
- **Izdavanje** — евидентира сваку позајмицу или резервацију; садржи датум, статус и напомену. Повезан са чланом и библиотекарском.
- **Stavkalzdavanja** — асоцијативна класа за везу M:N међу издавањем и књигама — омогућава да један захтев обухвати више књига.
- **Clan** — члан библиотеке; садржи JMBG, име, презиме, број телефона, корисничко име и лозинку. Повезан са градом.
- **Bibliotekar** — корисник са администраторским правима; садржи JMBG, име, презиме, корисничко име и лозинку.
- **Grad** — повезан са члановима (1:N). Може се унети ручно приликом регистрације ако не постоји у листи.



2.2 Списак функционалности

Функционалности за библиотекара:

- Преглед целокупног каталога књига са статусом сваке - доступна, резервисана или издата
- Додавање нове књиге у каталог заједно са аутором и жанром
- Измена детаља постојеће књиге (наслов, опис)
- Брисање књиге (уз проверу да тренутно није код неког члана)
- Преглед свих регистрованих чланова и броја активних задужења
- Преглед историје задужења сваког члана
- Одобравање резервација које су поднели чланови (прелаз из REZERVISANO у IZDATO)
- Брисање члана (уз проверу да нема активних задужења)

Функционалности за члана:

- Преглед доступних књига и таб са тренутно заузетим књигама
- Претрага књига по наслову
- Преглед детаља књиге — аутори, опис, статус доступности
- Слање захтева за позајмицу (резервација)
- Праћење статуса сопствених позајмица
- Враћање књиге

Опште функционалности:

- Регистрација новог члана са избором или уносом града
- Пријава за библиотекарe и чланове кроз јединствени endpoint - враћа JWT токен
- Аутоматска одјава при истеку токена

3. Спецификација

3.1 Архитектура система

Пројекат је реализован као двослојна апликација. Позадински слој је REST API прављен у ASP.NET Core, а кориснички интерфејс је React апликација написана у TypeScript-у. Комуникација међу слојевима иде искључиво кроз JSON формат. Аутентификација је заснована на JWT токенима - токен се чува у sessionStorage прегледача и аутоматски прикључује сваком захтеву кроз Axios interceptor.

3.2 Архитектурни обрасци

- **Repository Pattern** — сваки ентитет има свој репозиторијум који апстрахује приступ бази
- **Unit of Work** — координише трансакције над више репозиторијума кроз један SaveChanges позив
- **DTO слој** — одвојени Data Transfer Objects за улазне и излазне податке, доменски модел није директно изложен
- **Fluent Validation** — валидација улазних захтева издвојена из контролера у засебне класе валидатора

3.3 Технолошки склоп

Компонента	Технологија / библиотека
Backend framework	ASP.NET Core 10 Web API
Програмски језик	C#
ORM / миграције	Entity Framework Core 10 (Code-First, SQLite провајдер)
База података	SQLite
Аутентификација	JWT Bearer токени (System.IdentityModel.Tokens.Jwt)
Хешовање лозинки	BCrypt.Net-Next
Валидација	FluentValidation.AspNetCore
Frontend framework	React 18 + TypeScript (Vite)
HTTP клијент	Axios
CSS framework	Bootstrap 5

3.4 Структура backend пројекта

- **Biblioteka.Domain** — доменски модел: ентитети, интерфејси репозиторијума
- **Biblioteka.Infrastructure** — EF Core контекст, имплементације репозиторијума, миграције
- **Biblioteka.API** — контролери, DTO класе, валидатори, JWT сервис, Program.cs конфигурација

3.5 API руте

Метода	Рута	Опис
POST	/api/auth/login	Пријава (библиотекар или члан) — враћа JWT токен и податке о кориснику
GET	/api/knjige	Преузима све књиге из каталога
GET	/api/knjige/search/{naslov}	Претрага књига по наслову
GET	/api/knjige/{id}/autori	Аутори конкретне књиге
POST	/api/knjige	Додавање нове књиге заједно са аутором
PUT	/api/knjige/{id}	Измена податка о књизи
DELETE	/api/knjige/{id}	Брисање књиге из каталога
GET	/api/clanovi	Листа свих чланова
POST	/api/clanovi	Регистрација новог члана (без пријаве)
DELETE	/api/clanovi/{jmbg}	Брисање члана
GET	/api/izdavanja	Сва издавања — резервације и позајмице
POST	/api/izdavanja/rezervisi	Члан шаље захтев за резервацију
PUT	/api/izdavanja/{id}/odobri	Библиотекар одобрава резервацију
PUT	/api/izdavanja/{id}/vrati	Члан враћа позајмљену књигу
GET	/api/gradovi	Листа свих градова
POST	/api/gradovi	Додавање новог града

3.5 Додатна библиотека

У пројекту су коришћене две додатне библиотеке које нису саставни део стандардног ASP.NET Core пакета.

Hangfire

Hangfire је библиотека за управљање позадинским пословима у .NET апликацијама. Суштина је у томе да апликација може сама да извршава одређену логику у позадини, независно од тога да ли корисник тренутно нешто ради на систему — битно је само да је апликација покренута.

У оквиру овог пројекта, Hangfire обавља следећу улогу: аутоматски прати рокове резервација и позајмљених књига. Конкретно:

- Ако библиотекар не одобри резервацију у задатом времену, **Hangfire** аутоматски мења статус у бази на ODBIJENO
- Ако члан не врати књигу у задатом времену, **Hangfire** аутоматски мења статус на VRACENO

Поента ове функционалности је да се спречи ситуација у којој члан блокира књигу на неодређено дуго - пошто је у питању онлајн платформа, потребно је да систем сам реагује кад нико не реагује.

BCrypt.Net-Next

BCrypt.Net-Next је библиотека за хешовање лозинки. Приликом регистрације, лозинка корисника се никад не чува у отвореном облику у бази - уместо тога, BCrypt генерише хеш лозинку. Приликом пријаве, унета лозинка се упоређује са сачуваним хешом, а не са текстом лозинке директно. На овај начин, чак и ако неко добије приступ бази, лозинке корисника остају заштићене.

4. Имплементација

Комплетан изворни код пројекта доступан је на следећем GitHub репозиторијуму:

https://github.com/LukaDjokic14/biblioteka_.net.git

Репозиторијум садржи:

- **Biblioteka.sln** - Visual Studio solution са свим пројектима
- **README.md** - упутство за покретање
- **biblioteka-frontend/** - React/TypeScript frontend апликација

Покретање пројекта:

- Backend: отворити .sln у Visual Studio, покренути Update-Database у Package Manager Console, затим F5
- Frontend: `npm install`, па `npm run dev` у фолдеру `biblioteka-frontend/`